

[lick here to view code image](#)

```
In [5]: grades = [88, 75, 96, 55, 83]

In [6]: average(*grades)
Out[6]: 79.4
```

The call shown above is equivalent to `average(88, 75, 96, 55, 83)`.

4.12 METHODS: FUNCTIONS THAT BELONG TO OBJECTS

A **method** is simply a function that you call on an object using the form

```
object_name.method_name(arguments)
```

For example, the following session creates the string variable `s` and assigns it the string object `'Hello'`. Then the session calls the object's **lower** and **upper** methods, which produce *new* strings containing all-lowercase and all-uppercase versions of the original string, leaving `s` unchanged:

[lick here to view code image](#)

```
In [1]: s = 'Hello'

In [2]: s.lower()    # call lower    method on string object s
Out[2]: 'hello'

In [3]: s.upper()
Out[3]: 'HELLO'

In [4]: s
Out[4]: 'Hello'
```

The *Python Standard Library* reference at

```
https://docs.python.org/3/library/index.html
```

describes the methods of built-in types and the types in the Python Standard Library. In the “Object-Oriented Programming” chapter, you’ll create *custom* types called classes and define custom methods that you can call on objects of those classes.

4.13 SCOPE RULES

Each identifier has a **scope** that determines where you can use it in your program. For that portion of the program, the identifier is said to be “in scope.”

Local Scope

A local variable’s identifier has **local scope**. It’s “in scope” only from its definition to the end of the function’s block. It “goes out of scope” when the function returns to its caller. So, a local variable can be used only inside the function that defines it.

Global Scope

Identifiers defined outside any function (or class) have **global scope**—these may include functions, variables and classes. Variables with global scope are known as **global variables**. Identifiers with global scope can be used in a `.py` file or interactive session anywhere after they’re defined.

Accessing a Global Variable from a Function

You can access a global variable’s value inside a function:

[lick here to view code image](#)

```
In [1]: x = 7

In [2]: def access_global():
...:     print('x printed from   access_global:', x)
...:

In [3]: access_global()
x printed from access_global: 7
```

However, by default, you cannot *modify* a global variable in a function—when you first assign a value to a variable in a function’s block, Python creates a *new* local variable:

[lick here to view code image](#)

```
In [4]: def   try_to_modify_global():
...:     x = 3.5
...:     print('x printed from   try_to_modify_global:', x)
...:

In [5]: try_to_modify_global()
x printed from try_to_modify_global: 3.5
```

```
In [6]: x
Out[6]: 7
```

In function `try_to_modify_global`'s block, the local `x` **shadows** the global `x`, making it inaccessible in the scope of the function's block. Snippet [6] shows that global variable `x` still exists and has its original value (7) after function `try_to_modify_global` executes.

To modify a global variable in a function's block, you must use a **global** statement to declare that the variable is defined in the global scope:

[lick here to view code image](#)

```
In [7]: def modify_global():
...:     global x
...:     x = 'hello'
...:     print('x printed from modify_global:', x)
...:

In [8]: modify_global()
x printed from modify_global: hello

In [9]: x
Out[9]: 'hello'
```

Blocks vs. Suites

You've now defined function *blocks* and control statement *suites*. When you create a variable in a block, it's *local* to that block. However, when you create a variable in a control statement's suite, the variable's scope depends on where the control statement is defined:

- If the control statement is in the global scope, then any variables defined in the control statement have global scope.
- If the control statement is in a function's block, then any variables defined in the control statement have local scope.

We'll continue our scope discussion in the "Object-Oriented Programming" chapter when we introduce custom classes.

Shadowing Functions

In the preceding chapters, when summing values, we stored the sum in a variable named `total`. The reason we did this is that `sum` is a built-in function. If you define a variable named `sum`, it *shadows* the built-in function, making it inaccessible in your code. When you execute the following assignment, Python binds the identifier `sum` to the `int` object containing 15. At this point, the identifier `sum` no longer references the built-in function. So, when you try to use `sum` as a function, a `TypeError` occurs:

[lick here to view code image](#)

```
In [10]: sum = 10 + 5

In [11]: sum
Out[11]: 15

In [12]: sum([10, 5])
-----
TypeError                                 Traceback (most recent call last)
ipython-input-12-1237d97a65fb> in <module>()
----> 1 sum([10, 5])

TypeError: 'int' object is not callable
```

Statements at Global Scope

In the scripts you've seen so far, we've written some statements outside functions at the global scope and some statements inside function blocks. Script statements at global scope execute as soon as they're encountered by the interpreter, whereas statements in a block execute only when the function is called.

4.14 IMPORT: A DEEPER LOOK

You've imported modules (such as `math` and `random`) with a statement like:

```
import module_name
```

then accessed their features via each module's name and a dot (`.`). Also, you've imported a specific identifier from a module (such as the `decimal` module's `Decimal` type) with a statement like:

```
from module_name import identifier
```

then used that identifier without having to precede it with the module name and a dot (.).

Importing Multiple Identifiers from a Module

Using the `from import` statement you can import a comma-separated list of identifiers from a module then use them in your code without having to precede them with the module name and a dot (.):

[lick here to view code image](#)

```
In [1]: from math import ceil, floor

In [2]: ceil(10.3)
Out[2]: 11

In [3]: floor(10.7)
Out[3]: 10
```

Trying to use a function that's not imported causes a `NameError`, indicating that the name is not defined.

Caution: Avoid Wildcard Imports

You can import *all* identifiers defined in a module with a **wildcard import** of the form

```
from module_name import *
```

This makes all of the module's identifiers available for use in your code. Importing a module's identifiers with a wildcard `import` can lead to subtle errors—it's considered a dangerous practice that you should avoid. Consider the following snippets:

```
In [4]: e = 'hello'

In [5]: from math import *

In [6]: e
Out[6]: 2.718281828459045
```

Initially, we assign the string `'hello'` to a variable named `e`. After executing snippet [5] though, the variable `e` is replaced, possibly by accident, with the `math` module's constant `e`, representing the mathematical floating-point value e .

Binding Names for Modules and Module Identifiers

Sometimes it's helpful to import a module and use an abbreviation for it to simplify your code. The `import` statement's **as** clause allows you to specify the name used to reference the module's identifiers. For example, in [Section 3.14](#) we could have imported the `statistics` module and accessed its `mean` function as follows:

[lick here to view code image](#)

```
In [7]: import statistics as stats

In [8]: grades = [85, 93, 45, 87, 93]

In [9]: stats.mean(grades)
Out[9]: 80.6
```

As you'll see in later chapters, `import as` is frequently used to import Python libraries with convenient abbreviations, like `stats` for the `statistics` module. As another example, we'll use the `numpy` module which typically is imported with

```
import numpy as np
```

Library documentation often mentions popular shorthand names.

Typically, when importing a module, you should use `import` or `import as` statements, then access the module through the module name or the abbreviation following the `as` keyword, respectively. This ensures that you do not accidentally import an identifier that conflicts with one in your code.

4.15 PASSING ARGUMENTS TO FUNCTIONS: A DEEPER LOOK

Let's take a closer look at how arguments are passed to functions. In many programming languages, there are two ways to pass arguments—**pass-by-value** and **pass-by-reference** (sometimes called **call-by-value** and **call-by-reference**, respectively):

- With pass-by-value, the called function receives a *copy* of the argument's *value* and works exclusively with that copy. Changes to the function's copy do *not* affect the original variable's value in the caller.

- With pass-by-reference, the called function can access the argument’s value in the caller directly and modify the value if it’s mutable.

Python arguments are always passed by reference. Some people call this **pass-by-object-reference**, because “everything in Python is an object.”³ When a function call provides an argument, Python copies the argument object’s *reference*—not the object itself—into the corresponding parameter. This is important for performance. Functions often manipulate large objects—frequently copying them would consume large amounts of computer memory and significantly slow program performance.

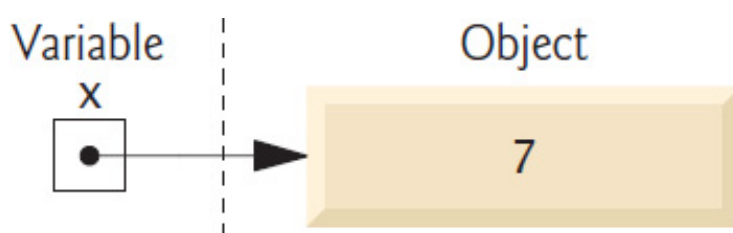
³ Even the functions you defined in this chapter and the classes (custom types) you’ll define in later chapters are objects in Python.

Memory Addresses, References and “Pointers”

You interact with an object via a reference, which behind the scenes is that object’s address (or location) in the computer’s memory—sometimes called a “pointer” in other languages. After an assignment like

```
x = 7
```

the variable `x` does not actually contain the value `7`. Rather, it contains a reference to an *object* containing `7` stored *elsewhere* in memory. You might say that `x` “points to” (that is, references) the object containing `7`, as in the diagram below:



Built-In Function `id` and Object Identities

Let’s consider how we pass arguments to functions. First, let’s create the integer variable `x` mentioned above—shortly we’ll use `x` as a function argument:

```
In [1]: x = 7
```

Now `x` refers to (or “points to”) the integer object containing `7`. No two separate objects can reside at the same address in memory, so every object in memory has a *unique address*. Though we can’t see an object’s address, we can use the built-in **`id` function**

to obtain a *unique* `int` value which identifies only that object while it remains in memory (you'll likely get a different value when you run this on your computer):

```
In [2]: id(x)
Out[2]: 4350477840
```

The integer result of calling `id` is known as the object's **identity**.⁴ No two objects in memory can have the same *identity*. We'll use object identities to demonstrate that objects are passed by reference.

⁴ According to the Python documentation, depending on the Python implementation you're using, an object's identity may be the object's actual memory address, but this is not required.

Passing an Object to a Function

Let's define a `cube` function that displays its parameter's identity, then returns the parameter's value cubed:

[lick here to view code image](#)

```
In [3]: def cube(number):
...:     print('id(number):', id(number))
...:     return number ** 3
...:
```

Next, let's call `cube` with the argument `x`, which refers to the integer object containing 7:

```
In [4]: cube(x)
id(number): 4350477840
Out[4]: 343
```

The identity displayed for `cube`'s parameter `number`—4350477840—is the *same* as that displayed for `x` previously. Since every object has a unique identity, both the *argument* `x` and the *parameter* `number` refer to the *same object* while `cube` executes. So when function `cube` uses its parameter `number` in its calculation, it gets the value of `number` from the original object in the caller.

Testing Object Identities with the `is` Operator

You also can prove that the argument and the parameter refer to the same object with Python's **is operator**, which returns `True` if its two operands have the *same identity*:

[lick here to view code image](#)

```
In [5]: def cube(number):
...:     print('number is x:',    number is x)  # x is a    global variab
...:     return number ** 3
...:

In [6]: cube(x)
number is x: True
Out[6]: 343
```

Immutable Objects as Arguments

When a function receives as an argument a reference to an *immutable* (unmodifiable) object—such as an `int`, `float`, `string` or `tuple`—even though you have direct access to the original object in the caller, you cannot modify the original immutable object's value. To prove this, first let's have `cube` display `id(number)` before and after assigning a new object to the parameter `number` via an augmented assignment:

[lick here to view code image](#)

```
In [7]: def cube(number):
...:     print('id(number) before    modifying number:', id(number))
...:     number **= 3
...:     print('id(number) after    modifying number:', id(number))
...:     return number
...:

In [8]: cube(x)
id(number) before modifying number: 4350477840
id(number) after  modifying number: 4396653744
Out[8]: 343
```

When we call `cube(x)`, the first `print` statement shows that `id(number)` initially is the same as `id(x)` in snippet [2]. Numeric values are immutable, so the statement

```
number **= 3
```

actually creates a *new object* containing the cubed value, then assigns that object's reference to parameter `number`. Recall that if there are no more references to the

original object, it will be *garbage collected*. Function `cube`'s second `print` statement shows the *new* object's identity. Object identities must be unique, so `number` must refer to a *different* object. To show that `x` was not modified, we display its value and identity again:

[lick here to view code image](#)

```
In [9]: print(f'x = {x}; id(x) = {id(x)}')
x = 7; id(x) = 4350477840
```

Mutable Objects as Arguments

In the next chapter, we'll show that when a reference to a *mutable* object like a list is passed to a function, the function *can* modify the original object in the caller.

4.16 RECURSION

Let's write a program to perform a famous mathematical calculation. Consider the *factorial* of a positive integer n , which is written $n!$ and pronounced " n factorial." This is the product

$$n \cdot (n - 1) \cdot (n - 2) \cdots 1$$

with $1!$ equal to 1 and $0!$ defined to be 1. For example, $5!$ is the product $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is equal to 120.

Iterative Factorial Approach

You can calculate $5!$ *iteratively* with a `for` statement, as in:

[lick here to view code image](#)

```
In [1]: factorial = 1

In [2]: for number in range(5, 0, -1):
...:     factorial *= number
...:

In [3]: factorial
Out[3]: 120
```

Recursive Problem Solving

Recursive problem-solving approaches have several elements in common. When you call a recursive function to solve a problem, it's actually capable of solving only the *simplest case(s)*, or **base case(s)**. If you call the function with a *base case*, it immediately returns a result. If you call the function with a more complex problem, it typically divides the problem into two pieces—one that the function knows how to do and one that it does not know how to do. To make recursion feasible, this latter piece must be a slightly simpler or smaller version of the original problem. Because this new problem resembles the original problem, the function calls a fresh *copy* of itself to work on the smaller problem—this is referred to as a **recursive call** and is also called the **recursion step**. This concept of separating the problem into two smaller portions is a form of the *divide-and-conquer* approach introduced earlier in the book.

The recursion step executes while the original function call is still active (i.e., it has not finished executing). It can result in many more recursive calls as the function divides each new subproblem into two conceptual pieces. For the recursion to eventually terminate, each time the function calls itself with a simpler version of the original problem, the sequence of smaller and smaller problems must *converge on a base case*. When the function recognizes the base case, it returns a result to the previous copy of the function. A sequence of returns ensues until the original function call returns the final result to the caller.

Recursive Factorial Approach

You can arrive at a recursive factorial representation by observing that $n!$ can be written as:

$$n! = n \cdot (n - 1)!$$

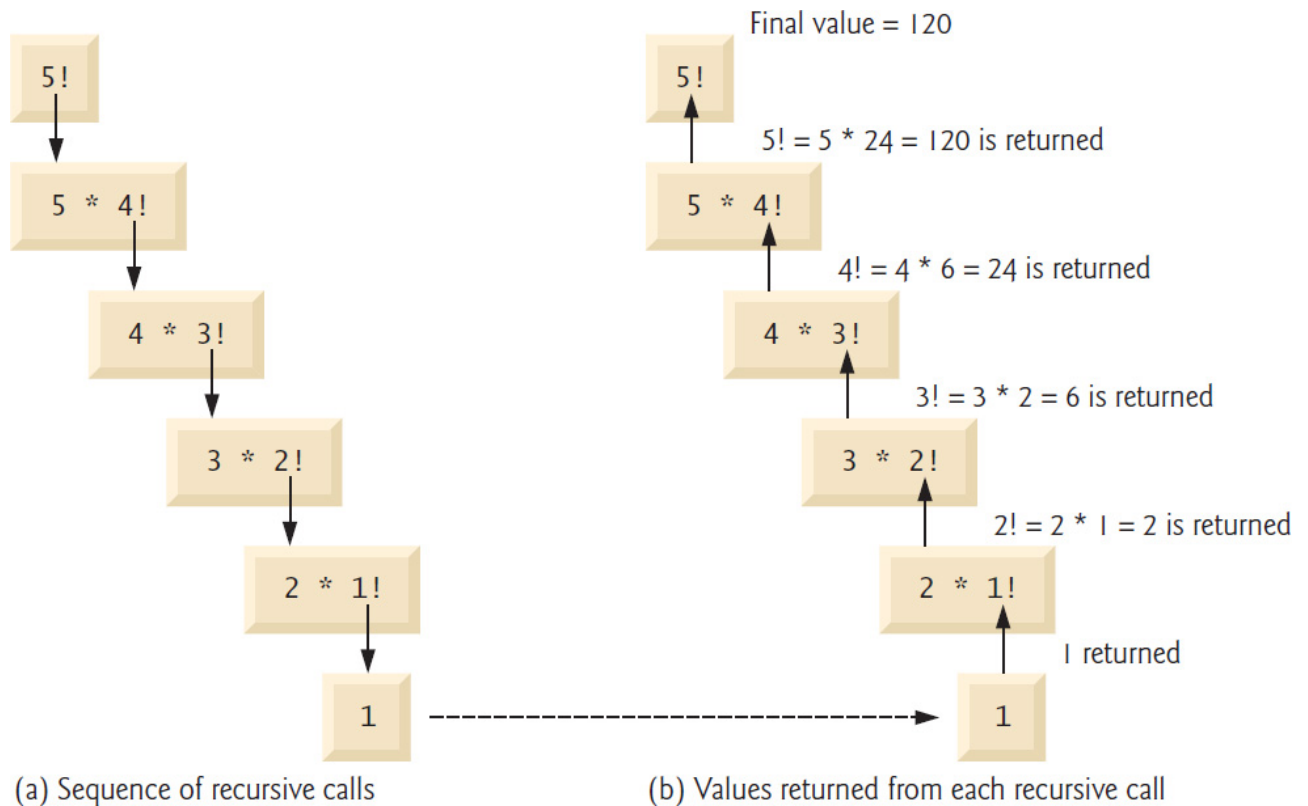
For example, $5!$ is equal to $5 \cdot 4!$, as in:

$$\begin{aligned} 5! &= 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \\ 5! &= 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1) \\ 5! &= 5 \cdot (4!) \end{aligned}$$

Visualizing Recursion

The evaluation of $5!$ would proceed as shown below. The left column shows how the succession of recursive calls proceeds until $1!$ (the base case) is evaluated to be 1, which terminates the recursion. The right column shows from bottom to top the values returned from each recursive call to its caller until the final value is calculated and

returned.



Implementing a Recursive Factorial Function

The following session uses recursion to calculate and display the factorials of the integers 0 through 10:

[lick here to view code image](#)

```
In [4]: def factorial(number):
...:     """Return factorial of number."""
...:     if number <= 1:
...:         return 1
...:     return number * factorial(number - 1) # recursive call
...:

In [5]: for i in range(11):
...:     print(f'{i}! = {factorial(i)}')
...:

0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

Snippet [4]’s recursive function `factorial` first determines whether the *terminating condition* `number <= 1` is `True`. If this condition is `True` (the base case), `factorial` returns 1 and no further recursion is necessary. If `number` is greater than 1, the second `return` statement expresses the problem as the product of `number` and a *recursive call* to `factorial` that evaluates `factorial(number - 1)`. This is a slightly smaller problem than the original calculation, `factorial(number)`. Note that function `factorial` must receive a *nonnegative* argument. We do not test for this case.

The loop in snippet [5] calls the `factorial` function for the values from 0 through 10. The output shows that factorial values grow quickly. *Python does not limit the size of an integer*, unlike many other programming languages.

Indirect Recursion

A recursive function may call another function, which may, in turn, make a call back to the recursive function. This is known as an **indirect recursive call** or **indirect recursion**. For example, function A calls function B, which makes a call back to function A. This is still recursion because the second call to function A is made while the first call to function A is active. That is, the first call to function A has not yet finished executing (because it is waiting on function B to return a result to it) and has not returned to function A’s original caller.

Stack Overflow and Infinite Recursion

Of course, the amount of memory in a computer is finite, so only a certain amount of memory can be used to store activation records on the function-call stack. If more recursive function calls occur than can have their activation records stored on the stack, a fatal error known as **stack overflow** occurs. This typically is the result of **infinite recursion**, which can be caused by omitting the base case or writing the recursion step incorrectly so that it does not converge on the base case. This error is analogous to the problem of an *infinite loop* in an *iterative* (nonrecursive) solution.

4.17 FUNCTIONAL-STYLE PROGRAMMING

Like other popular languages, such as Java and C#, Python is not a purely functional language. Rather, it offers “functional-style” features that help you write code which is less likely to contain errors, more concise and easier to read, debug and modify.

Functional-style programs also can be easier to parallelize to get better performance on

today's multi-core processors. The chart below lists most of Python's key functional-style programming capabilities and shows in parentheses the chapters in which we initially cover many of them.

Functional-style programming topics		
avoiding side effects (4)	generator functions	
closures	higher-order	lazy evaluation (5)
declarative programming (4)	functions (5)	list comprehensions (5)
decorators (10)	immutability (4)	operator module (5, 11, 16)
dictionary comprehensions (6)	internal iteration (4)	pure functions (4)
filter/map/reduce (5)	iterators (3)	range function (3, 4)
functools module	itertools module (16)	reductions (3, 5)
generator expressions (5)	lambda expressions (5)	set comprehensions (6)

We cover most of these features throughout the book—many with code examples and others from a literacy perspective. You've already used list, string and built-in function *range* *iterators* with the `for` statement, and several *reductions* (functions `sum`, `len`, `min` and `max`). We discuss declarative programming, immutability and internal iteration below.

What vs. How

As the tasks you perform get more complicated, your code can become harder to read, debug and modify, and more likely to contain errors. Specifying *how* the code works can become complex.

Functional-style programming lets you simply say *what* you want to do. It hides many

etails of *how* to perform each task. Typically, library code handles the *how* for you. As you'll see, this can eliminate many errors.

Consider the `for` statement in many other programming languages. Typically, *you* must specify all the details of counter-controlled iteration: a control variable, its initial value, how to increment it and a loop-continuation condition that uses the control variable to determine whether to continue iterating. This style of iteration is known as **external iteration** and is error-prone. For example, you might provide an incorrect initializer, increment or loop-continuation condition. External iteration **mutates** (that is, modifies) the control variable, and the `for` statement's suite often mutates other variables as well. Every time you modify variables you could introduce errors. Functional-style programming emphasizes **immutability**. That is, it avoids operations that modify variables' values. We'll say more in the next chapter.

Python's `for` statement and `range` function *hide* most counter-controlled iteration details. You specify *what* values `range` should produce and the variable that should receive each value as it's produced. Function `range` *knows how* to produce those values. Similarly, the `for` statement *knows how* to get each value from `range` and *how* to stop iterating when there are no more values. Specifying *what*, but not *how*, is an important aspect of **internal iteration**—a key functional-style programming concept.

The Python built-in functions `sum`, `min` and `max` each use internal iteration. To total the elements of the list `grades`, you simply declare *what* you want to do—that is, `sum(grades)`. Function `sum` *knows how* to iterate through the list and add each element to the running total. Stating what you *want* done rather than programming *how* to do it is known as **declarative programming**.

Pure Functions

In pure functional programming language you focus on writing pure functions. A **pure function's** result depends only on the argument(s) you pass to it. Also, given a particular argument (or arguments), a pure function always produces the same result. For example, built-in function `sum`'s return value depends only on the iterable you pass to it. Given a list `[1, 2, 3]`, `sum` *always* returns 6 no matter how many times you call it. Also, a pure function does not have *side effects*. For example, even if you pass a *mutable* list to a pure function, the list will contain the same values before and after the function call. When you call the pure function `sum`, it does not modify its argument.

[lick here to view code image](#)

```
In [1]: values = [1, 2, 3]

In [2]: sum(values)
Out[2]: 6

In [3]: sum(values)    # same call    always returns same result
Out[3]: 6

In [4]: values
Out[5]: [1, 2, 3]
```

In the next chapter, we'll continue using functional-style programming concepts. Also, you'll see that *functions are objects* that you can pass to other functions as data.

4.18 INTRO TO DATA SCIENCE: MEASURES OF DISPERSION

In our discussion of descriptive statistics, we've considered the measures of central tendency—mean, median and mode. These help us categorize typical values in a group—such as the mean height of your classmates or the most frequently purchased car brand (the mode) in a given country.

When we're talking about a group, the entire group is called the **population**. Sometimes a population is quite large, such as the people likely to vote in the next U.S. presidential election, which is a number in excess of 100,000,000 people. For practical reasons, the polling organizations trying to predict who will become the next president work with carefully selected small subsets of the population known as **samples**. Many of the polls in the 2016 election had sample sizes of about 1000 people.

In this section, we continue discussing basic descriptive statistics. We introduce **measures of dispersion** (also called **measures of variability**) that help you understand how spread out the values are. For example, in a class of students, there may be a bunch of students whose height is close to the average, with smaller numbers of students who are considerably shorter or taller.

For our purposes, we'll calculate each measure of dispersion both by hand and with functions from the module `statistics`, using the following population of 10 six-sided die rolls:

```
1, 3, 4, 2, 6, 5, 3, 4, 5, 2
```

Variance

To determine the **variance**,⁵ we begin with the mean of these values—3.5. You obtain this result by dividing the sum of the face values, 35, by the number of rolls, 10. Next, we subtract the mean from every die value (this produces some negative results):

⁵ For simplicity, we're calculating the *population variance*. There is a subtle difference between the *population variance* and the *sample variance*. Instead of dividing by n (the number of die rolls in our example), sample variance divides by $n - 1$. The difference is pronounced for small samples and becomes insignificant as the sample size increases. The `statistics` module provides the functions `pvariance` and `variance` to calculate the population variance and sample variance, respectively. Similarly, the `statistics` module provides the functions `pstdev` and `stdev` to calculate the population standard deviation and sample standard deviation, respectively.

```
-2.5, -0.5, 0.5, -1.5, 2.5, 1.5, -0.5, 0.5, 1.5, -1.5
```

Then, we square each of these results (yielding only positives):

```
6.25, 0.25, 0.25, 2.25, 6.25, 2.25, 0.25, 0.25, 2.25, 2.25
```

Finally, we calculate the mean of these squares, which is 2.25 ($22.5 / 10$)—this is the **population variance**. Squaring the difference between each die value and the mean of all die values emphasizes **outliers**—the values that are farthest from the mean. As we get deeper into data analytics, sometimes we'll want to pay careful attention to outliers, and sometimes we'll want to ignore them. The following code uses the `statistics` module's **pvariance** function to confirm our manual result:

[lick here to view code image](#)

```
In [1]: import statistics

In [2]: statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[2]: 2.25
```

Standard Deviation

The **standard deviation** is the square root of the variance (in this case, 1.5), which tones down the effect of the outliers. The smaller the variance and standard deviation

are, the closer the data values are to the mean and the less overall **dispersion** (that is, **spread**) there is between the values and the mean. The following code calculates the **population standard deviation** with the statistics module's **pstdev** function, confirming our manual result:

[lick here to view code image](#)

```
In [3]: statistics.pstdev([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[3]: 1.5
```

Passing the `pvariance` function's result to the `math` module's `sqrt` function confirms our result of 1.5:

[lick here to view code image](#)

```
In [4]: import math

In [5]: math.sqrt(statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2]))
Out[5]: 1.5
```

Advantage of Population Standard Deviation vs. Population Variance

Suppose you've recorded the March Fahrenheit temperatures in your area. You might have 31 numbers such as 19, 32, 28 and 35. The units for these numbers are degrees. When you square your temperatures to calculate the population variance, the units of the population variance become "degrees squared." When you take the square root of the population variance to calculate the population standard deviation, the units once again become degrees, which are the *same* units as your temperatures.

4.19 WRAP-UP

In this chapter, we created custom functions. We imported capabilities from the `random` and `math` modules. We introduced random-number generation and used it to simulate rolling a six-sided die. We packed multiple values into tuples to return more than one value from a function. We also unpacked a tuple to access its values. We discussed using the Python Standard Library's modules to avoid "reinventing the wheel."

We created functions with default parameter values and called functions with keyword arguments. We also defined functions with arbitrary argument lists. We called methods of objects. We discussed how an identifier's scope determines where in your program

ou can use it.

We presented more about importing modules. You saw that arguments are passed-by-reference to functions, and how the function-call stack and stack frames support the function-call-and-return mechanism. We also presented a recursive function and began introducing Python’s functional-style programming capabilities. We’ve introduced basic list and tuple capabilities over the last two chapters—in the next chapter, we’ll discuss them in detail.

Finally, we continued our discussion of descriptive statistics by introducing measures of dispersion—variance and standard deviation—and calculating them with functions from the Python Standard Library’s `statistics` module.

For some types of problems, it’s useful to have functions call themselves. A **recursive function** calls itself, either directly or indirectly through another function.

5. Sequences: Lists and Tuples

Objectives

In this chapter, you'll:

- Create and initialize lists and tuples.
- Refer to elements of lists, tuples and strings.
- Sort and search lists, and search tuples.
- Pass lists and tuples to functions and methods.
- Use list methods to perform common manipulations, such as searching for items, sorting a list, inserting items and removing items.
- Use additional Python functional-style programming capabilities, including lambdas and the functional-style programming operations filter, map and reduce.
- Use functional-style list comprehensions to create lists quickly and easily, and use generator expressions to generate values on demand.
- Use two-dimensional lists.
- Enhance your analysis and presentation skills with the Seaborn and Matplotlib visualization libraries.

Outline

.1 Introduction

.2 Lists

.3 Tuples

.4 Unpacking Sequences

.5 Sequence Slicing

.6 del Statement

.7 Passing Lists to Functions

.8 Sorting Lists

.9 Searching Sequences

.10 Other List Methods

.11 Simulating Stacks with Lists

.12 List Comprehensions

.13 Generator Expressions

.14 Filter, Map and Reduce

.15 Other Sequence Processing Functions

.16 Two-Dimensional Lists

.17 Intro to Data Science: Simulation and Static Visualizations

.17.1 Sample Graphs for 600, 60,000 and 6,000,000 Die Rolls

.17.2 Visualizing Die-Roll Frequencies and Percentages

.18 Wrap-Up

5.1 INTRODUCTION

In the last two chapters, we briefly introduced the list and tuple sequence types for representing ordered collections of items. **Collections** are prepackaged data structures consisting of related data items. Examples of collections include your favorite songs on your smartphone, your contacts list, a library's books, your cards in a card game, your favorite sports team's players, the stocks in an investment portfolio, patients in a cancer study and a shopping list. Python's built-in collections enable you to store and access

data conveniently and efficiently. In this chapter, we discuss lists and tuples in more detail.

We'll demonstrate common list and tuple manipulations. You'll see that lists (which are modifiable) and tuples (which are not) have many common capabilities. Each can hold items of the same or different types. Lists can **dynamically resize** as necessary, growing and shrinking at execution time. We discuss one-dimensional and two-dimensional lists.

In the preceding chapter, we demonstrated random-number generation and simulated rolling a six-sided die. We conclude this chapter with our next Intro to Data Science section, which uses the visualization libraries Seaborn and Matplotlib to interactively develop static bar charts containing the die frequencies. In the next chapter's Intro to Data Science section, we'll present an animated visualization in which the bar chart changes *dynamically* as the number of die rolls increases—you'll see the law of large numbers “in action.”

5.2 LISTS

Here, we discuss lists in more detail and explain how to refer to particular list **elements**. Many of the capabilities shown in this section apply to all sequence types.

Creating a List

Lists typically store **homogeneous data**, that is, values of the *same* data type. Consider the list `c`, which contains five integer elements:

[lick here to view code image](#)

```
In [1]: c = [-45, 6, 0, 72, 1543]

In [2]: c
Out[2]: [-45, 6, 0, 72, 1543]
```

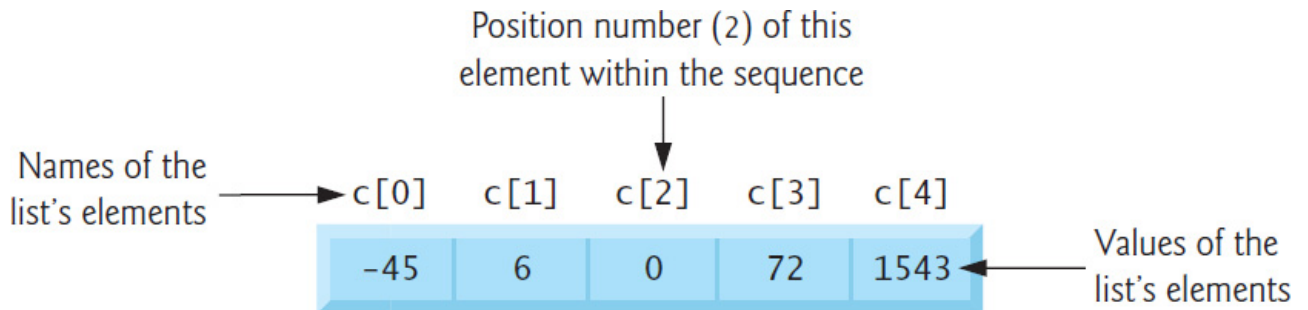
They also may store **heterogeneous data**, that is, data of many different types. For example, the following list contains a student's first name (a string), last name (a string), grade point average (a `float`) and graduation year (an `int`):

[lick here to view code image](#)

```
['Mary', 'Smith', 3.57, 2022]
```

Accessing Elements of a List

You reference a list element by writing the list's name followed by the element's **index** (that is, its **position number**) enclosed in square brackets (`[]`), known as the **subscription operator**. The following diagram shows the list `c` labeled with its element names:



The first element in a list has the index 0. So, in the five-element list `c`, the first element is named `c[0]` and the last is `c[4]`:

```
In [3]: c[0]
Out[3]: -45
```

```
In [4]: c[4]
Out[4]: 1543
```

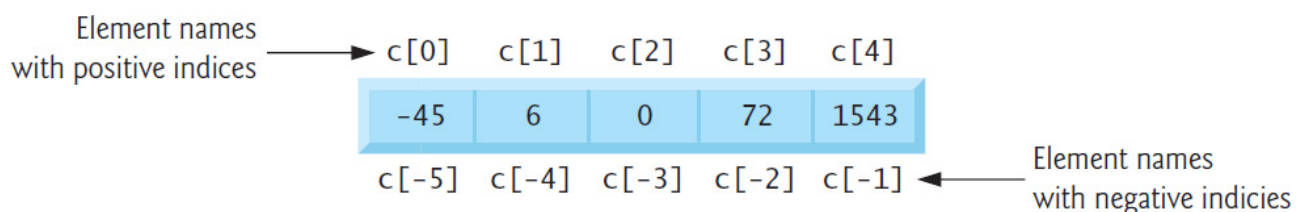
Determining a List's Length

To get a list's length, use the built-in **len function**:

```
In [5]: len(c)
Out[5]: 5
```

Accessing Elements from the End of the List with Negative Indices

Lists also can be accessed from the end by using *negative indices*:



So, list `c`'s last element (`c[4]`), can be accessed with `c[-1]` and its first element with `c[-5]`:

```
In [6]: c[-1]
Out[6]: 1543
```

```
In [7]: c[-5]
Out[7]: -45
```

Indices Must Be Integers or Integer Expressions

An index must be an integer or integer expression (or a *slice*, as we'll soon see):

```
In [8]: a = 1

In [9]: b = 2

In [10]: c[a + b]
Out[10]: 72
```

Using a non-integer index value causes a `TypeError`.

Lists Are Mutable

Lists are mutable—their elements can be modified:

[lick here to view code image](#)

```
In [11]: c[4] = 17

In [12]: c
Out[12]: [-45, 6, 0, 72, 17]
```

You'll soon see that you also can insert and delete elements, changing the list's length.

Some Sequences Are Immutable

Python's string and tuple sequences are immutable—they cannot be modified. You can get the individual characters in a string, but attempting to assign a new value to one of the characters causes a `TypeError`:

[lick here to view code image](#)

```
n [13]: s = 'hello'

In [14]: s[0]
Out[14]: 'h'

In [15]: s[0] = 'H'
```

```
TypeError                                Traceback (most recent call last)
<ipython-input-15-812ef2514689> in <module>()
----> 1 s[0] = 'H'

TypeError: 'str' object does not support item assignment
```

Attempting to Access a Nonexistent Element

Using an out-of-range list, tuple or string index causes an `IndexError`:

[lick here to view code image](#)

```
In [16]: c[100]

-----
IndexError                                Traceback (most recent call last)
ipython-input-16-9a31ea1e1a13> in <module>()
----> 1 c[100]

IndexError: list index out of range
```

Using List Elements in Expressions

List elements may be used as variables in expressions:

```
In [17]: c[0] + c[1] + c[2]
Out[17]: -39
```

Appending to a List with `+=`

Let's start with an *empty* list `[]`, then use a `for` statement and `+=` to append the values 1 through 5 to the list—the list grows dynamically to accommodate each item:

[lick here to view code image](#)

```
In [18]: a_list = []

In [19]: for number in range(1, 6):
...:     a_list += [number]
...:

In [20]: a_list
Out[20]: [1, 2, 3, 4, 5]
```

When the left operand of `+=` is a list, the right operand must be an *iterable*; otherwise, a `TypeError` occurs. In snippet [19]’s suite, the square brackets around `number` create a one-element list, which we append to `a_list`. If the right operand contains multiple elements, `+=` appends them all. The following appends the characters of `'Python'` to the list `letters`:

[lick here to view code image](#)

```
In [21]: letters = []

In [22]: letters += 'Python'

In [23]: letters
Out[23]: ['P', 'y', 't', 'h', 'o', 'n']
```

If the right operand of `+=` is a tuple, its elements also are appended to the list. Later in the chapter, we’ll use the list method `append` to add items to a list.

Concatenating Lists with +

You can **concatenate** two lists, two tuples or two strings using the `+` operator. The result is a *new* sequence of the same type containing the left operand’s elements followed by the right operand’s elements. The original sequences are unchanged:

[lick here to view code image](#)

```
In [24]: list1 = [10, 20, 30]

In [25]: list2 = [40, 50]

In [26]: concatenated_list = list1 + list2

In [27]: concatenated_list
Out[27]: [10, 20, 30, 40, 50]
```

A `TypeError` occurs if the `+` operator’s operands are difference sequence types—for example, concatenating a list and a tuple is an error.

Using `for` and `range` to Access List Indices and Values

List elements also can be accessed via their indices and the subscription operator (`[]`):

[lick here to view code image](#)

```
In [28]: for i in range(len(concatenated_list)):
...:     print(f'{i}: {concatenated_list[i]}')
...:
0: 10
1: 20
2: 30
3: 40
4: 50
```

The function call `range(len(concatenated_list))` produces a sequence of integers representing `concatenated_list`'s indices (in this case, 0 through 4). When looping in this manner, you must ensure that indices remain in range. Soon, we'll show a safer way to access element indices and values using built-in function `enumerate`.

Comparison Operators

You can compare entire lists element-by-element using comparison operators:

[lick here to view code image](#)

```
In [29]: a = [1, 2, 3]

In [30]: b = [1, 2, 3]

In [31]: c = [1, 2, 3, 4]

In [32]: a == b # True: corresponding elements in both are equal
Out[32]: True

In [33]: a == c # False: a and c have different elements and lengths
Out[33]: False

In [34]: a < c # True: a has fewer elements than c
Out[34]: True

In [35]: c >= b # True: elements 0-2 are equal but c has more elements
Out[35]: True
```

5.3 TUPLES

As discussed in the preceding chapter, tuples are immutable and typically store heterogeneous data, but the data can be homogeneous. A tuple's length is its number of elements and cannot change during program execution.

Creating Tuples

o create an empty tuple, use empty parentheses:

```
In [1]: student_tuple = ()

In [2]: student_tuple
Out[2]: ()

In [3]: len(student_tuple)
Out[3]: 0
```

Recall that you can pack a tuple by separating its values with commas:

[lick here to view code image](#)

```
In [4]: student_tuple = 'John', 'Green', 3.3

In [5]: student_tuple
Out[5]: ('John', 'Green', 3.3)

In [6]: len(student_tuple)
Out[6]: 3
```

When you output a tuple, Python always displays its contents in parentheses. You may surround a tuple’s comma-separated list of values with optional parentheses:

[lick here to view code image](#)

```
In [7]: another_student_tuple = ('Mary', 'Red', 3.3)

In [8]: another_student_tuple
Out[8]: ('Mary', 'Red', 3.3)
```

The following code creates a one-element tuple:

[lick here to view code image](#)

```
In [9]: a_singleton_tuple = ('red',) # note the comma

In [10]: a_singleton_tuple
Out[10]: ('red',)
```

The comma (,) that follows the string 'red' identifies `a_singleton_tuple` as a tuple—the parentheses are optional. If the comma were omitted, the parentheses would

be redundant, and `a_singleton_tuple` would simply refer to the string `'red'` rather than a tuple.

Accessing Tuple Elements

A tuple's elements, though related, are often of multiple types. Usually, you do not iterate over them. Rather, you access each individually. Like list indices, tuple indices start at 0. The following code creates `time_tuple` representing an hour, minute and second, displays the tuple, then uses its elements to calculate the number of seconds since midnight—note that we perform a *different* operation with each value in the tuple:

[lick here to view code image](#)

```
In [11]: time_tuple = (9, 16, 1)

In [12]: time_tuple
Out[12]: (9, 16, 1)

In [13]: time_tuple[0] * 3600 + time_tuple[1] * 60 + time_tuple[2]
Out[13]: 33361
```

Assigning a value to a tuple element causes a `TypeError`.

Adding Items to a String or Tuple

As with lists, the `+=` augmented assignment statement can be used with strings and tuples, even though they're *immutable*. In the following code, after the two assignments, `tuple1` and `tuple2` refer to the *same* tuple object:

```
In [14]: tuple1 = (10, 20, 30)

In [15]: tuple2 = tuple1

In [16]: tuple2
Out[16]: (10, 20, 30)
```

Concatenating the tuple `(40, 50)` to `tuple1` creates a *new* tuple, then assigns a reference to it to the variable `tuple1`—`tuple2` still refers to the original tuple:

[lick here to view code image](#)

```
In [17]: tuple1 += (40, 50)
```

```
In [18]: tuple1
Out[18]: (10, 20, 30, 40, 50)

In [19]: tuple2
Out[19]: (10, 20, 30)
```

For a string or tuple, the item to the right of += must be a string or tuple, respectively—mixing types causes a `TypeError`.

Appending Tuples to Lists

You can use += to append a tuple to a list:

[lick here to view code image](#)

```
In [20]: numbers = [1, 2, 3, 4, 5]

In [21]: numbers += (6, 7)

In [22]: numbers
Out[22]: [1, 2, 3, 4, 5, 6, 7]
```

Tuples May Contain Mutable Objects

Let's create a `student_tuple` with a first name, last name and list of grades:

[lick here to view code image](#)

```
In [23]: student_tuple = ('Amanda', 'Blue', [98, 75, 87])
```

Even though the tuple is immutable, its list element is mutable:

[lick here to view code image](#)

```
In [24]: student_tuple[2][1] = 85

In [25]: student_tuple
Out[25]: ('Amanda', 'Blue', [98, 85, 87])
```

In the *double-subscripted name* `student_tuple[2][1]`, Python views `student_tuple[2]` as the element of the tuple containing the list `[98, 75, 87]`, then uses `[1]` to access the list element containing 75. The assignment in snippet `[24]`

replaces that grade with 85.

5.4 UNPACKING SEQUENCES

The previous chapter introduced tuple unpacking. You can unpack any sequence's elements by assigning the sequence to a comma-separated list of variables. A `ValueError` occurs if the number of variables to the left of the assignment symbol is not identical to the number of elements in the sequence on the right:

[lick here to view code image](#)

```
In [1]: student_tuple = ('Amanda', [98, 85, 87])

In [2]: first_name, grades = student_tuple

In [3]: first_name
Out[3]: 'Amanda'

In [4]: grades
Out[4]: [98, 85, 87]
```

The following code unpacks a string, a list and a sequence produced by `range`:

[lick here to view code image](#)

```
In [5]: first, second = 'hi'

In [6]: print(f'{first} {second}')
h i

In [7]: number1, number2, number3 = [2, 3, 5]

In [8]: print(f'{number1} {number2} {number3}')
2 3 5

In [9]: number1, number2, number3 = range(10, 40, 10)

In [10]: print(f'{number1} {number2} {number3}')
10 20 30
```

Swapping Values Via Packing and Unpacking

You can swap two variables' values using sequence packing and unpacking:

[lick here to view code image](#)

```
In [11]: number1 = 99

In [12]: number2 = 22

In [13]: number1, number2 = (number2, number1)

In [14]: print(f'number1 = {number1}; number2 = {number2}')
number1 = 22; number2 = 99
```

Accessing Indices and Values Safely with Built-in Function `enumerate`

Earlier, we called `range` to produce a sequence of index values, then accessed list elements in a `for` loop using the index values and the subscription operator (`[]`). This is error-prone because you could pass the wrong arguments to `range`. If any value produced by `range` is an out-of-bounds index, using it as an index causes an `IndexError`.

The preferred mechanism for accessing an element's index *and* value is the built-in function `enumerate`. This function receives an iterable and creates an iterator that, for each element, returns a tuple containing the element's index and value. The following code uses the built-in function `list` to create a list containing `enumerate`'s results:

[lick here to view code image](#)

```
In [15]: colors = ['red', 'orange', 'yellow']

In [16]: list(enumerate(colors))
Out[16]: [(0, 'red'), (1, 'orange'), (2, 'yellow')]
```

Similarly the built-in function `tuple` creates a tuple from a sequence:

[lick here to view code image](#)

```
In [17]: tuple(enumerate(colors))
Out[17]: ((0, 'red'), (1, 'orange'), (2, 'yellow'))
```

The following `for` loop unpacks each tuple returned by `enumerate` into the variables `index` and `value` and displays them:

[lick here to view code image](#)

```
In [18]: for index, value in enumerate(colors):
```

```

...:     print(f'{index}: {value}')
...:
0: red
1: orange
2: yellow

```

Creating a Primitive Bar Chart

The following script creates a primitive **bar chart** where each bar's length is made of asterisks (*) and is proportional to the list's corresponding element value. We use the function `enumerate` to access the list's indices and values safely. To run this example, change to this chapter's `ch05 examples` folder, then enter:

```
ipython fig05_01.py
```

or, if you're in IPython already, use the command:

```
run fig05_01.py
```

[lick here to view code image](#)

```

1 # fig05_01.py
2 """Displaying a bar chart"""
3 numbers = [19, 3, 15, 7, 11]
4
5 print('\nCreating a bar chart from numbers:')
6 print(f'Index{"Value":>8}    Bar')
7
8 for index, value in enumerate(numbers):
9     print(f'{index:>5}{value:>8}    {"*" * value}')

```

[lick here to view code image](#)

```

Creating a bar chart from numbers:
Index    Value    Bar
   0      19    *****
   1       3     ***
   2      15    *****
   3       7     *****
   4      11    *****

```

The `for` statement uses `enumerate` to get each element's index and value, then

displays a formatted line containing the index, the element value and the corresponding bar of asterisks. The expression

```
"*" * value
```

creates a string consisting of `value` asterisks. When used with a sequence, the multiplication operator (`*`) *repeats* the sequence—in this case, the string `"*"`—`value` times. Later in this chapter, we'll use the open-source Seaborn and Matplotlib libraries to display a publication-quality bar chart visualization.

5.5 SEQUENCE SLICING

You can **slice** sequences to create new sequences of the same type containing *subsets* of the original elements. Slice operations can modify mutable sequences—those that do *not* modify a sequence work identically for lists, tuples and strings.

Specifying a Slice with Starting and Ending Indices

Let's create a slice consisting of the elements at indices 2 through 5 of a list:

[lick here to view code image](#)

```
In [1]: numbers = [2, 3, 5, 7, 11, 13, 17, 19]

In [2]: numbers[2:6]
Out[2]: [5, 7, 11, 13]
```

The slice *copies* elements from the *starting index* to the left of the colon (2) up to, but not including, the *ending index* to the right of the colon (6). The original list is not modified.

Specifying a Slice with Only an Ending Index

If you omit the starting index, 0 is assumed. So, the slice `numbers[:6]` is equivalent to the slice `numbers[0:6]`:

[lick here to view code image](#)

```
In [3]: numbers[:6]
Out[3]: [2, 3, 5, 7, 11, 13]

In [4]: numbers[0:6]
```

```
Out[4]: [2, 3, 5, 7, 11, 13]
```

Specifying a Slice with Only a Starting Index

If you omit the ending index, Python assumes the sequence's length (8 here), so snippet [5]'s slice contains the elements of `numbers` at indices 6 and 7:

[lick here to view code image](#)

```
In [5]: numbers[6:]  
Out[5]: [17, 19]  
  
In [6]: numbers[6:len(numbers)]  
Out[6]: [17, 19]
```

Specifying a Slice with No Indices

Omitting both the start and end indices copies the entire sequence:

[lick here to view code image](#)

```
In [7]: numbers[:]  
Out[7]: [2, 3, 5, 7, 11, 13, 17, 19]
```

Though slices create new objects, slices make **shallow copies** of the elements—that is, they copy the elements' references but not the objects they point to. So, in the snippet above, the new list's elements refer to the *same objects* as the original list's elements, rather than to separate copies. In the “Array-Oriented Programming with NumPy” chapter, we'll explain *deep* copying, which actually copies the referenced objects themselves, and we'll point out when deep copying is preferred.

Slicing with Steps

The following code uses a *step* of 2 to create a slice with every other element of `numbers`:

```
In [8]: numbers[::2]  
Out[8]: [2, 5, 11, 17]
```

We omitted the start and end indices, so 0 and `len(numbers)` are assumed, respectively.

Slicing with Negative Indices and Steps

You can use a negative step to select slices in *reverse* order. The following code concisely creates a new list in reverse order:

[lick here to view code image](#)

```
In [9]: numbers[::-1]
Out[9]: [19, 17, 13, 11, 7, 5, 3, 2]
```

This is equivalent to:

[lick here to view code image](#)

```
In [10]: numbers[-1:-9:-1]
Out[10]: [19, 17, 13, 11, 7, 5, 3, 2]
```

Modifying Lists Via Slices

You can modify a list by assigning to a slice of it—the rest of the list is unchanged. The following code replaces `numbers`' first three elements, leaving the rest unchanged:

[lick here to view code image](#)

```
In [11]: numbers[0:3] = ['two', 'three', 'five']

In [12]: numbers
Out[12]: ['two', 'three', 'five', 7, 11, 13, 17, 19]
```

The following deletes only the first three elements of `numbers` by assigning an *empty* list to the three-element slice:

[lick here to view code image](#)

```
In [13]: numbers[0:3] = []

In [14]: numbers
Out[14]: [7, 11, 13, 17, 19]
```

The following assigns a list's elements to a slice of every other element of `numbers`:

[lick here to view code image](#)

```
In [15]: numbers = [2, 3, 5, 7, 11, 13, 17, 19]

In [16]: numbers[::2] = [100, 100, 100, 100]

In [17]: numbers
Out[17]: [100, 3, 100, 7, 100, 13, 100, 19]

In [18]: id(numbers)
Out[18]: 4434456648
```

Let's delete all the elements in `numbers`, leaving the *existing* list empty:

[lick here to view code image](#)

```
In [19]: numbers[:] = []

In [20]: numbers
Out[20]: []

In [21]: id(numbers)
Out[21]: 4434456648
```

Deleting `numbers`' contents (snippet [19]) is different from assigning `numbers` a *new* empty list `[]` (snippet [22]). To prove this, we display `numbers`' identity after each operation. The identities are different, so they represent separate objects in memory:

```
In [22]: numbers = []

In [23]: numbers
Out[23]: []

In [24]: id(numbers)
Out[24]: 4406030920
```

When you assign a new object to a variable (as in snippet [21]), the original object will be garbage collected if no other variables refer to it.

5.6 DEL STATEMENT

The **`del` statement** also can be used to remove elements from a list and to delete variables from the interactive session. You can remove the element at any valid index or the element(s) from any valid slice.

Deleting the Element at a Specific List Index

et's create a list, then use `del` to remove its last element:

[lick here to view code image](#)

```
In [1]: numbers = list(range(0, 10))

In [2]: numbers
Out[2]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

In [3]: del numbers[-1]

In [4]: numbers
Out[4]: [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Deleting a Slice from a List

The following deletes the list's first two elements:

```
In [5]: del numbers[0:2]

In [6]: numbers
Out[6]: [2, 3, 4, 5, 6, 7, 8]
```

The following uses a step in the slice to delete every other element from the entire list:

```
In [7]: del numbers[::2]

In [8]: numbers
Out[8]: [3, 5, 7]
```

Deleting a Slice Representing the Entire List

The following code deletes all of the list's elements:

```
In [9]: del numbers[:]

In [10]: numbers
Out[10]: []
```

Deleting a Variable from the Current Session

The `del` statement can delete any variable. Let's delete `numbers` from the interactive session, then attempt to display the variable's value, causing a `NameError`:

[lick here to view code image](#)

```
In [11]: del numbers

In [12]: numbers
-----
NameError                                Traceback (most recent call last)
ipython-input-12-426f8401232b> in <module>()
----> 1 numbers

NameError: name 'numbers' is not defined
```

5.7 PASSING LISTS TO FUNCTIONS

In the last chapter, we mentioned that all objects are passed by reference and demonstrated passing an immutable object as a function argument. Here, we discuss references further by examining what happens when a program passes a mutable list object to a function.

Passing an Entire List to a Function

Consider the function `modify_elements`, which receives a reference to a list and multiplies each of the list's element values by 2:

[lick here to view code image](#)

```
In [1]: def modify_elements(items):
...:     """Multiplies all element values in items by 2."""
...:     for i in range(len(items)):
...:         items[i] *= 2
...:

In [2]: numbers = [10, 3, 7, 1, 9]

In [3]: modify_elements(numbers)

In [4]: numbers
Out[4]: [20, 6, 14, 2, 18]
```

Function `modify_elements`'s `items` parameter receives a reference to the *original* list, so the statement in the loop's suite modifies each element in the original list object.

Passing a Tuple to a Function

When you pass a tuple to a function, attempting to modify the tuple's immutable elements results in a `TypeError`:

[lick here to view code image](#)

```
In [5]: numbers_tuple = (10, 20, 30)

In [6]: numbers_tuple
Out[6]: (10, 20, 30)

In [7]: modify_elements(numbers_tuple)
-----
TypeError                                Traceback (most recent call last)
ipython-input-7-9339741cd595> in <module>()
----> 1 modify_elements(numbers_tuple)

<ipython-input-1-27acb8f8f44c> in modify_elements(items)
      2     """Multiplies all element values in items by 2."""
      3     for i in range(len(items)):
----> 4         items[i] *= 2
      5
      6

TypeError: 'tuple' object does not support item assignment
```

Recall that tuples may contain mutable objects, such as lists. Those objects still can be modified when a tuple is passed to a function.

A Note Regarding Tracebacks

The previous traceback shows the *two* snippets that led to the `TypeError`. The first is snippet [7]'s function call. The second is snippet [1]'s function definition. Line numbers precede each snippet's code. We've demonstrated mostly single-line snippets. When an exception occurs in such a snippet, it's always preceded by `----> 1`, indicating that line 1 (the snippet's only line) caused the exception. Multiline snippets like the definition of `modify_elements` show consecutive line numbers starting at 1. The notation `----> 4` above indicates that the exception occurred in line 4 of `modify_elements`. No matter how long the traceback is, the last line of code with `---->` caused the exception.

5.8 SORTING LISTS

Sorting enables you to arrange data either in ascending or descending order.

Sorting a List in Ascending Order

List method `sort` *modifies* a list to arrange its elements in ascending order:

[lick here to view code image](#)

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [2]: numbers.sort()

In [3]: numbers
Out[3]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Sorting a List in Descending Order

To sort a list in descending order, call list method `sort` with the optional keyword argument `reverse`—set to `True` (`False` is the default):

[lick here to view code image](#)

```
In [4]: numbers.sort(reverse=True)

In [5]: numbers
Out[5]: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Built-In Function `sorted`

Built-in function `sorted` *returns a new list* containing the sorted elements of its argument *sequence*—the original sequence is *unmodified*. The following code demonstrates function `sorted` for a list, a string and a tuple:

[lick here to view code image](#)

```
In [6]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [7]: ascending_numbers = sorted(numbers)

In [8]: ascending_numbers
Out[8]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

In [9]: numbers
Out[9]: [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [10]: letters = 'fadgchjebi'

In [11]: ascending_letters = sorted(letters)
```

```
In [12]: ascending_letters
Out[12]: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

In [13]: letters
Out[13]: 'fadgchjebi'

In [14]: colors = ('red', 'orange', 'yellow', 'green', 'blue')

In [15]: ascending_colors = sorted(colors)

In [16]: ascending_colors
Out[16]: ['blue', 'green', 'orange', 'red', 'yellow']

In [17]: colors
Out[17]: ('red', 'orange', 'yellow', 'green', 'blue')
```

Use the optional keyword argument `reverse` with the value `True` to sort the elements in descending order.

5.9 SEARCHING SEQUENCES

Often, you'll want to determine whether a sequence (such as a list, tuple or string) contains a value that matches a particular **key** value. **Searching** is the process of locating a key.

List Method `index`

List method `index` takes as an argument a search key—the value to locate in the list—then searches through the list from index 0 and returns the index of the *first* element that matches the search key:

[lick here to view code image](#)

```
In [1]: numbers = [3, 7, 1, 4, 2, 8, 5, 6]

In [2]: numbers.index(5)
Out[2]: 6
```

A `ValueError` occurs if the value you're searching for is not in the list.

Specifying the Starting Index of a Search

Using method `index`'s optional arguments, you can search a subset of a list's elements. You can use `*` to *multiply a sequence*—that is, append a sequence to itself multiple

times. After the following snippet, `numbers` contains two copies of the original list's contents:

[lick here to view code image](#)

```
In [3]: numbers *= 2

In [4]: numbers
Out[4]: [3, 7, 1, 4, 2, 8, 5, 6, 3, 7, 1, 4, 2, 8, 5, 6]
```

The following code searches the updated list for the value 5 starting from index 7 and continuing through the end of the list:

```
In [5]: numbers.index(5, 7)
Out[5]: 14
```

Specifying the Starting and Ending Indices of a Search

Specifying the starting and ending indices causes `index` to search from the starting index up to but not including the ending index location. The call to `index` in snippet [5]:

```
numbers.index(5, 7)
```

assumes the length of `numbers` as its optional third argument and is equivalent to:

[lick here to view code image](#)

```
numbers.index(5, 7, len(numbers))
```

The following looks for the value 7 in the range of elements with indices 0 through 3:

[lick here to view code image](#)

```
In [6]: numbers.index(7, 0, 4)
Out[6]: 1
```

Operators `in` and `not in`

Operator `in` tests whether its right operand's iterable contains the left operand's value:

```
In [7]: 1000 in numbers
Out[7]: False

In [8]: 5 in numbers
Out[8]: True
```

Similarly, operator `not in` tests whether its right operand's iterable does *not* contain the left operand's value:

```
In [9]: 1000 not in numbers
Out[9]: True

In [10]: 5 not in numbers
Out[10]: False
```

Using Operator `in` to Prevent a `ValueError`

You can use the operator `in` to ensure that calls to method `index` do not result in `ValueErrors` for search keys that are not in the corresponding sequence:

[lick here to view code image](#)

```
In [11]: key = 1000

In [12]: if key in numbers:
...:     print(f'found {key}    at index {numbers.index(search_key)}')
...: else:
...:     print(f'{key} not found')
...:
1000 not found
```

Built-In Functions `any` and `all`

Sometimes you simply need to know whether *any* item in an iterable is `True` or whether *all* the items are `True`. The built-in function `any` returns `True` if any item in its iterable argument is `True`. The built-in function `all` returns `True` if all items in its iterable argument are `True`. Recall that nonzero values are `True` and `0` is `False`. Non-empty iterable objects also evaluate to `True`, whereas any empty iterable evaluates to `False`. Functions `any` and `all` are additional examples of internal iteration in functional-style programming.

5.10 OTHER LIST METHODS

Lists also have methods that add and remove elements. Consider the list

`color_names`:

[lick here to view code image](#)

```
In [1]: color_names = ['orange', 'yellow', 'green']
```

Inserting an Element at a Specific List Index

Method **`insert`** adds a new item at a specified index. The following inserts 'red' at index 0:

[lick here to view code image](#)

```
In [2]: color_names.insert(0, 'red')

In [3]: color_names
Out[3]: ['red', 'orange', 'yellow', 'green']
```

Adding an Element to the End of a List

You can add a new item to the end of a list with method **`append`**:

[lick here to view code image](#)

```
In [4]: color_names.append('blue')

In [5]: color_names
Out[5]: ['red', 'orange', 'yellow', 'green', 'blue']
```

Adding All the Elements of a Sequence to the End of a List

Use list method **`extend`** to add all the elements of another sequence to the end of a list:

[lick here to view code image](#)

```
In [6]: color_names.extend(['indigo', 'violet'])

In [7]: color_names
Out[7]: ['red', 'orange', 'yellow', 'green', 'blue', 'indigo', 'violet']
```

This is the equivalent of using `+=`. The following code adds all the characters of a string then all the elements of a tuple to a list:

[lick here to view code image](#)

```
In [8]: sample_list = []

In [9]: s = 'abc'

In [10]: sample_list.extend(s)

In [11]: sample_list
Out[11]: ['a', 'b', 'c']

In [12]: t = (1, 2, 3)

In [13]: sample_list.extend(t)

In [14]: sample_list
Out[14]: ['a', 'b', 'c', 1, 2, 3]
```

Rather than creating a temporary variable, like `t`, to store a tuple before appending it to a list, you might want to pass a tuple directly to `extend`. In this case, the tuple's parentheses are required, because `extend` expects one iterable argument:

[lick here to view code image](#)

```
In [15]: sample_list.extend((4, 5, 6)) # note the extra parentheses

In [16]: sample_list
Out[16]: ['a', 'b', 'c', 1, 2, 3, 4, 5, 6]
```

A `TypeError` occurs if you omit the required parentheses.

Removing the First Occurrence of an Element in a List

Method `remove` deletes the first element with a specified value—a `ValueError` occurs if `remove`'s argument is not in the list:

[lick here to view code image](#)

```
In [17]: color_names.remove('green')

In [18]: color_names
Out[18]: ['red', 'orange', 'yellow', 'blue', 'indigo', 'violet']
```

Emptying a List

To delete all the elements in a list, call method **clear**:

```
In [19]: color_names.clear()

In [20]: color_names
Out[20]: []
```

This is the equivalent of the previously shown slice assignment

```
color_names[:] = []
```

Counting the Number of Occurrences of an Item

List method **count** searches for its argument and returns the number of times it is found:

[lick here to view code image](#)

```
In [21]: responses = [1, 2, 5, 4, 3, 5, 2, 1, 3, 3,
...:                  1, 4, 3, 3, 3, 2, 3, 3, 2, 2]
...:

In [22]: for i in range(1, 6):
...:     print(f'{i} appears {responses.count(i)} times in responses')
...:

1 appears 3 times in responses
2 appears 5 times in responses
3 appears 8 times in responses
4 appears 2 times in responses
5 appears 2 times in responses
```

Reversing a List's Elements

List method **reverse** reverses the contents of a list in place, rather than creating a reversed copy, as we did with a slice previously:

[lick here to view code image](#)

```
In [23]: color_names = ['red', 'orange', 'yellow', 'green', 'blue']
```

```
In [24]: color_names.reverse()

In [25]: color_names
Out[25]: ['blue', 'green', 'yellow', 'orange', 'red']
```

Copying a List

List method `copy` returns a *new* list containing a *shallow* copy of the original list:

[lick here to view code image](#)

```
In [26]: copied_list = color_names.copy()

In [27]: copied_list
Out[27]: ['blue', 'green', 'yellow', 'orange', 'red']
```

This is equivalent to the previously demonstrated slice operation:

```
copied_list = color_names[:]
```

5.11 SIMULATING STACKS WITH LISTS

The preceding chapter introduced the function-call stack. Python does not have a built-in stack type, but you can think of a stack as a constrained list. You *push* using list method `append`, which adds a new element to the *end* of the list. You *pop* using list method `pop` with no arguments, which removes and returns the item at the *end* of the list.

Let's create an empty list called `stack`, push (`append`) two strings onto it, then `pop` the strings to confirm they're retrieved in last-in, first-out (LIFO) order:

[lick here to view code image](#)

```
In [1]: stack = []

In [2]: stack.append('red')

In [3]: stack
Out[3]: ['red']

In [4]: stack.append('green')

In [5]: stack
Out[5]: ['red', 'green']
```

```
In [6]: stack.pop()
Out[6]: 'green'

In [7]: stack
Out[7]: ['red']

In [8]: stack.pop()
Out[8]: 'red'

In [9]: stack
Out[9]: []

In [10]: stack.pop()
-----
IndexError                                Traceback (most recent call last)
<ipython-input-10-50ea7ec13fbe> in <module>()
----> 1 stack.pop()

IndexError: pop from empty list
```

or each `pop` snippet, the value that `pop` removes and returns is displayed. Popping from an empty stack causes an `IndexError`, just like accessing a nonexistent list element with `[]`. To prevent an `IndexError`, ensure that `len(stack)` is greater than 0 before calling `pop`. You can run out of memory if you keep pushing items faster than you pop them.

You also can use a list to simulate another popular collection called a **queue** in which you insert at the back and delete from the front. Items are retrieved from queues in **first-in, first-out (FIFO) order**.

5.12 LIST COMPREHENSIONS

Here, we continue discussing *functional-style* features with **list comprehensions**—a concise and convenient notation for creating new lists. List comprehensions can replace many `for` statements that iterate over existing sequences and create new lists, such as:

[lick here to view code image](#)

```
In [1]: list1 = []

In [2]: for item in range(1, 6):
...:     list1.append(item)
...:

In [3]: list1
```

```
Out[3]: [1, 2, 3, 4, 5]
```

Using a List Comprehension to Create a List of Integers

We can accomplish the same task in a single line of code with a list comprehension:

[lick here to view code image](#)

```
In [4]: list2 = [item for item in range(1, 6)]

In [5]: list2
Out[5]: [1, 2, 3, 4, 5]
```

Like snippet [2]’s `for` statement, the list comprehension’s **for clause**

```
for item in range(1, 6)
```

iterates over the sequence produced by `range(1, 6)`. For each `item`, the list comprehension evaluates the expression to the left of the `for` clause and places the expression’s value (in this case, the `item` itself) in the new list. Snippet [4]’s particular comprehension could have been expressed more concisely using the function `list`:

```
list2 = list(range(1, 6))
```

Mapping: Performing Operations in a List Comprehension’s Expression

A list comprehension’s expression can perform tasks, such as calculations, that **map** elements to new values (possibly of different types). Mapping is a common functional-style programming operation that produces a result with the *same* number of elements as the original data being mapped. The following comprehension maps each value to its cube with the expression `item ** 3`:

[lick here to view code image](#)

```
In [6]: list3 = [item ** 3 for item in range(1, 6)]

In [7]: list3
Out[7]: [1, 8, 27, 64, 125]
```

Filtering: List Comprehensions with `if` Clauses